

Contents

CLAUDIU user manual	1
Quick start	1
Features	1
Configuration	2
How the status strip works	3
Worked example: adding a project and a snippet	3
Shortcuts	4
REST/WS API	4
Command line	5
Logs	5
Rebuilding this manual	5
Launching from inside Claude Code	5
Known limitations	5

CLAUDIU user manual

CLAUDIU (working codename) is a local, browser-based tabbed interface for running several Claude Code sessions at once. A small Python server spawns real **claude** CLI processes in pseudo-terminals and streams them into browser tabs via xterm.js, so every tab behaves exactly like a real terminal — same colors, same prompts, same keybindings — while the browser layer adds tabs, a calmer theme, adjustable fonts, snippets, and a project launcher on top.

Quick start

```
pip install -e .
python -m claudiu
```

or, on Windows, double-click `run_claudiu.bat`. The server starts on `http://127.0.0.1:8642/` (127.0.0.1 only — never reachable from other machines) and opens it in your default browser automatically.

Features

- **Tabs** — one tab per live **claude** session, named after the project folder and renamable.
- **Terminal fidelity** — each tab embeds a real interactive **claude** process rendered by xterm.js: skills, plugins, slash commands, and permission prompts all behave exactly as in a plain terminal.
- **Session survival on refresh** — sessions live on the server, not in the browser tab; refreshing or closing Chrome never kills them.
- **Reconnect with backoff** — a dropped websocket retries automatically with backoff behind a visible “reconnecting...” strip; it never fails silently.
- **Resume after crash** — after a browser or PC crash, a resume list offers one-click **claude --resume <session-id>** per recent project.
- **Snippets bar + palette** — frequently used prompt text as buttons under the tab strip and in a fuzzy-searchable palette (**Ctrl+k**).
- **Project launcher** — configured working folders (plus a browse option) in the new-session dialog; one click starts **claude** in the chosen directory.
- **Per-tab font size** — each tab’s font size is adjustable and remembered independently.
- **Scrollbar search** — search within a tab’s capped scrollbar, with highlighting.
- **Rename by double-click** — double-click a tab’s label to rename it.
- **Status strip** — above each terminal, a one-line box shows the last prompt you typed in that session (click it to expand long prompts). Each tab carries a small light — pulsing amber while Claude is working, a faster accent-coloured pulse when it is waiting for your input on a permission prompt, and steady green once it is done — and a context gauge that fills as the conversation grows and shifts from green through amber to red as it approaches the point where auto-compaction starts hurting. A tab whose turn finishes

while you are looking elsewhere gets the “unseen” dot. All three read Claude Code’s own session transcript; nothing is guessed from terminal output (see “How the status strip works” below).

- **Light / dark / system theme** — a theme button in the tab bar cycles the interface between following your OS (**system**), **light**, and **dark**; the choice is remembered per browser. Set the default with `ui_theme`. (The terminal keeps its own **theme** colours in every mode.)
- **Redesigned launcher** — the new-session window opens with “Any folder” on top (type a path, pick a recently used one from the dropdown, or **Browse** the filesystem and **New folder** to create one), and two independently scrolling columns below: your configured **Projects** and **Resume recent**.
- **Jump to latest** — when you scroll up in a tab, a “↓ latest” button appears to return to the live output.
- **Calm dark theme** — near-black background, warm-gray foreground, muted ANSI colors, steady non-blinking cursor; every color is a config value. Sessions start with Claude Code’s **dark-ansi** theme so *all* of its colours come from that palette (Claude Code’s default themes print hard-coded truecolor that no terminal palette can soften); bold text keeps its colour instead of jumping to the bright variant.

Configuration

One human-editable JSON file. Default location `~/.claudiu/config.json`, created with defaults the first time the server runs if it does not exist yet. Override the path with the `CLAUDIU_CONFIG` environment variable, or with the `--config` flag on the command line. Keys the file does not mention fall back to the defaults below; keys the file has that this version of CLAUDIU does not recognize are kept as-is and reported as a startup warning, never rejected.

Key	Default	Meaning
<code>port</code>	8642	TCP port the server listens on (bound to 127.0.0.1 only)
<code>claude_command</code>	<code>["claude"]</code>	argv used to spawn each session
<code>replay_chunks</code>	2000	max buffered output <i>chunks</i> per session kept for replay on (re)connect (each chunk up to 64 KiB; see below)
<code>scrollback_lines</code>	10000	scrollback cap shown in the browser terminal
<code>font_size</code>	16	default terminal font size in pixels
<code>font_family</code>	<code>"Consolas, 'Cascadia Mono', monospace"</code>	terminal font stack
<code>theme</code>	a 16-color dark theme object	background, foreground, cursor, selection background, and the 16 ANSI colors
<code>claude_theme</code>	<code>"dark-ansi"</code>	Claude Code theme passed to every spawned session via <code>--settings</code> ; "" leaves Claude Code’s own setting alone (its default themes bypass theme with truecolor)
<code>status_poll_ms</code>	2000	how often the page refreshes the status strip, light and gauge
<code>context_window_tokens</code>	200000	the model’s context window, denominator of the context gauge
<code>context_warn_pct</code>	50	gauge turns amber at this percentage of the context window
<code>context_danger_pct</code>	75	gauge turns red at this percentage (must be <code>context_warn_pct</code>)
<code>ui_theme</code>	<code>"system"</code>	interface theme: <code>"system"</code> , <code>"light"</code> , or <code>"dark"</code> (the tab-bar button overrides this per browser)

Key	Default	Meaning
<code>permission_patterns</code>	four common prompts	substrings that mark a session as “waiting for you” when they appear in its terminal output (ANSI stripped, case-insensitive)
<code>recent_max</code>	15	how many recently launched folders to remember
<code>shortcuts</code>	see the Shortcuts table below	interface keyboard shortcut map
<code>projects</code>	<code>[]</code>	launcher entries: <code>{"name", "path", "args": []}</code>
<code>snippets</code>	<code>[]</code>	snippet-bar entries: <code>{"name", "text", "send": false}</code>

`replay_chunks` caps the number of buffered output *chunks*, not bytes or lines: each chunk is up to 64 KiB (the pty is read in 64 KiB blocks), so the worst-case memory a single session’s replay buffer can hold is `replay_chunks` × 64 KiB — 2000 × 64 KiB 125 MiB at the default. Raise or lower `replay_chunks` to trade replay depth against memory per session.

How the status strip works

Claude Code writes every session’s transcript to `~/.claude/projects/<project-slug>/<session-id>.jsonl` as it runs. CLAUDIU starts each session with an explicit `--session-id`, so it knows which file belongs to which tab, and reads only the *tail* of that file (a few hundred KB at most) every `status_poll_ms`:

- **last prompt** — the transcript’s `last-prompt` record (or the last user message you typed, never text injected by skills or hooks);
- **busy / ready / waiting** — the last assistant record’s `stop_reason`: a finished turn means ready; a pending tool call, or a prompt with no answer yet, means busy. When the session is busy *and* its terminal shows one of the `permission_patterns` (a “Do you want to proceed?” prompt), the state becomes **waiting** — the transcript has no record for a blocked prompt, so this one signal is read from the terminal output rather than the transcript;
- **context gauge** — the last assistant record’s token usage (input + cache-creation + cache-read), divided by `context_window_tokens`.

Sessions started with `--continue` have no id CLAUDIU can know in advance; their tab shows a dim light and no gauge. Sessions resumed with `--resume <id>` use that id. If the transcript cannot be found or read, the strip stays empty and the light stays dim — the terminal itself is never affected.

Worked example: adding a project and a snippet

Edit `~/.claudiu/config.json` (or the file `CLAUDIU_CONFIG` points at) and add to the `projects` and `snippets` lists:

```
{
  "projects": [
    {"name": "My App", "path": "C:/code/my-app", "args": ["--continue"]}
  ],
  "snippets": [
    {"name": "run tests", "text": "run the test suite and fix any failures",
     "send": true}
  ]
}
```

Restart the server to pick up the change (config is loaded once at startup; refreshing the page alone does not reload it). “My App” now appears in the project launcher, and “run tests” appears as a button in the snippets bar and in the `Ctrl+k` palette; because `send` is `true`, choosing it types the text and presses Enter.

Shortcuts

Interface-level keyboard shortcuts, all remappable via the `shortcuts` config key. They never intercept keys the CLI itself needs (plain `Ctrl+C`, `Ctrl+R`, arrow keys, and so on).

Action	Default key
<code>tab_1</code>	<code>Alt+1</code> — switch to tab 1
<code>tab_2</code>	<code>Alt+2</code> — switch to tab 2
<code>tab_3</code>	<code>Alt+3</code> — switch to tab 3
<code>tab_4</code>	<code>Alt+4</code> — switch to tab 4
<code>tab_5</code>	<code>Alt+5</code> — switch to tab 5
<code>tab_6</code>	<code>Alt+6</code> — switch to tab 6
<code>tab_7</code>	<code>Alt+7</code> — switch to tab 7
<code>tab_8</code>	<code>Alt+8</code> — switch to tab 8
<code>tab_9</code>	<code>Alt+9</code> — switch to tab 9
<code>tab_prev</code>	<code>Alt+ArrowLeft</code> — previous tab
<code>tab_next</code>	<code>Alt+ArrowRight</code> — next tab
<code>new_session</code>	<code>Alt+t</code> — open the new-session launcher
<code>close_tab</code>	<code>Alt+w</code> — close the active tab (asks; killing the session needs explicit confirmation)
<code>font_bigger</code>	<code>Alt+=</code> — increase this tab's font size
<code>font_smaller</code>	<code>Alt+-</code> — decrease this tab's font size
<code>font_reset</code>	<code>Alt+0</code> — reset this tab's font size
<code>search</code>	<code>Ctrl+Shift+f</code> — search this tab's scrollback
<code>snippet_palette</code>	<code>Ctrl+k</code> — open the fuzzy-searchable snippet palette
<code>help</code>	<code>Alt+h</code> — show the shortcut list (also the <code>?</code> button in the tab bar)

You do not need this table open while working: the `?` button at the right of the tab bar (or `Alt+h`) shows the live bindings — including any you remapped — every tab's tooltip names its switch key, and the new-session launcher's footer repeats the two you need most.

Why Alt-based defaults: Chrome reserves `Ctrl+T`, `Ctrl+W`, `Ctrl+Tab`, and `Ctrl+1...Ctrl+9` at the browser level — a web page cannot intercept them — so none of those combinations can ever be a working interface default; CLAUDIU's defaults use `Alt` instead. For the same reason, do not rebind any shortcut to bare `Escape`: the shortcut handler runs in the capture phase and would suppress the dialog-closing `Escape` handler that the launcher and rename dialogs rely on.

REST/WS API

Route	Description
<code>GET /api/sessions</code>	list live sessions
<code>POST /api/sessions</code>	create a session (body: path, args, title)
<code>PATCH /api/sessions/<id></code>	rename a session (body: title)
<code>DELETE /api/sessions/<id></code>	kill a session
<code>GET /api/config</code>	effective config plus load warnings
<code>GET /api/resume</code>	recent resumable Claude sessions
<code>GET /api/status</code>	per-session status: last prompt, busy/ready, context use
<code>GET /api/recent</code>	recently launched folders (<code>~/.claudiu/recent.json</code>)
<code>GET /api/dirs</code>	list sub-directories of a path (launcher folder picker)
<code>POST /api/mkdir</code>	create a folder (body: <code>parent</code> , <code>name</code>)
<code>WS /ws/<id></code>	terminal stream (terminado protocol)

GET /api/resume scans the real ~/.claude/projects directory of the user running the server — the resume list you see is your own Claude Code history on that machine, in that account. Each project entry includes an exists flag (true/false) so the launcher can skip projects whose directory has since moved or been deleted.

Command line

```
python -m claudiu [--config PATH] [--port PORT] [--log-dir DIR]
                  [--no-browser] [--verbose | --quiet] [--version]
```

Flag	Help
--config	path to config.json (default: ~/.claudiu/config.json, or \$CLAUDIU_CONFIG)
--port	override the config port (config default: 8642)
--log-dir	directory for audit logs (default: <config dir>/logs)
--no-browser	do not open the browser automatically
--verbose	debug output on the console
--quiet	warnings and errors only on the console
--version	print the version and exit
--help	print the argument summary and exit

Logs

Every run writes an audit log to <config dir>/logs/claudiu-<timestamp>.log (override the directory with --log-dir) recording: the command line invoked, CLAUDIU/Python/Tornado/Terminado versions, any config load warnings, session start/rename/kill events, and how the run ended. Console verbosity is independent of the log file: --verbose prints debug lines to the console too, --quiet restricts the console to warnings and errors; the log file itself is always written at debug level.

Rebuilding this manual

This document is the source of truth; USER_MANUAL.html (and USER_MANUAL.pdf, when the tools are available) are built from it and committed so readers need no tooling of their own. To regenerate them after editing this file:

```
python docs/build_manual.py
```

Uses pandoc for the HTML and pandoc + xelatex for the PDF when both are on PATH; otherwise falls back to a small stdlib-only Markdown renderer for the HTML and prints that the PDF was skipped. Always exits 0. --src and --outdir override the input file and output directory.

Launching from inside Claude Code

If you start claudiu from a Claude Code session (a shell tool, a hook, a sub-agent), the server inherits that session's environment, and every claude it spawned would otherwise inherit it too — the CLI then treats itself as a nested child session and turns transcript saving off (“Transcript saving is off — inherited CLAUDE_CODE_CHILD_SESSION marker”). CLAUDIU strips the nesting markers (CLAUDECODE, CLAUDE_CODE_CHILD_SESSION, CLAUDE_CODE_SESSION_ID, CLAUDE_PID, the messaging/bridge/entrypoint/execpath variables) from each session's environment, so sessions started here are ordinary top-level sessions with normal transcript saving and --resume behaviour. User configuration such as CLAUDE_CODE_MAX_OUTPUT_TOKENS is passed through untouched.

Known limitations

- **Windows-first.** Development and hand-testing happen on Windows. Linux and macOS are exercised only by CI (unit, integration, and end-to-end tests all run there too) — they are not hand-tested.

- **One browser page per server.** The server assumes a single open page; it is not designed for multiple browser tabs/windows pointed at the same running instance.
- **No split panes.** One session per interface tab; no multi-pane layouts.
- **No remote access.** The server binds `127.0.0.1` only — it is not reachable from other devices, and there is no plan to make it so. Requests must also originate from the page itself: a foreign `Origin` or `Host` header (CSRF, or DNS rebinding to `127.0.0.1`) is refused with 403.
- **No transcript export.** CLAUDIU does not render or export session transcripts; that is the job of the separate `claude-session-publisher` tool.
- **localStorage use is minimal.** The only thing CLAUDIU stores in the browser's `localStorage` is each tab's remembered font size — nothing else about a session lives in the browser; the browser is a disposable view onto server-side sessions.
- **The status strip is read from Claude Code's transcript files.** It depends on their current record shapes (`last-prompt`, `stop_reason`, `usage`); a future Claude Code release that changes them degrades the strip to “unknown” (dim light, no gauge, no prompt) — never to a wrong answer, and never to a broken terminal.
- **Keystrokes during a reconnect gap are dropped.** If you type while a tab's websocket is reconnecting (the “reconnecting...” strip is visible), those keystrokes are not queued and are lost — wait for the strip to clear before typing.